

Assignment 9

Author: Lan Stare
Date: 12. 11. 2025

Question 1

ADS, Ass 9

①

length	1	2	3	4	5
price	8	20	29	38	49

$f(j) = \begin{cases} 0 & j=0 \\ \max\{p_i + f(j-i) \mid 1 \leq i \leq j\} & j>0 \end{cases}$

table = []

for i in range(j+1):

if i == 0:

table.append(f(i), 0)

else:

table.append(f(i), f(i))

It really build from 0 to j (bottomup)

builds a table for every f(i)

table: [(f(0), 0), (f(1), 8), (f(2), 20), (f(3), 29), (f(4), 40), (f(5), 49)]

type	value
5 x 1	5 x 8 = 40
3 x 1 + 1 x 2	3 x 8 + 1 x 20 = 44
1 x 2 + 2 x 2	1 x 8 + 2 x 20 = 48
2 x 1 + 1 x 3	2 x 8 + 1 x 29 = 45
1 x 2 + 1 x 3	1 x 20 + 1 x 29 = 49
1 x 1 + 1 x 4	1 x 8 + 1 x 38 = 46
1 x 5	1 x 49 = 49

max value = 49

Question 2

②

0 h_1 h_2 ... h_i ... h_n

$p[j]$ $(200 - (h_n - h_j))^2$

$p(n)$... min penalty for travelling to hotel j

$p[n] = \begin{cases} 0 & n=0 \\ \min\{p[j] + (200 - (h_n - h_j))^2 \mid 0 \leq j < n\} & j>0 \end{cases}$

EXPLANATION: At lenght h_n we choose a random hotel j (on the path of h_n) at which we stop. We then need to calculate the best option for travelling to that hotel j and add the penalty for travelling from hotel j to hotel n in one day. By going through all hotels on the path from 1 to n, we each time find the optimal solution for stopping at hotel j right before hotel n. An optimal algorithm would remember if the overall minimum changes when a person visits a particular hotel, meaning that if the overall penalty is smaller for hotel j - 1, that hotel (j - 1) would replace hotel j.

BOTTOM-UP AND COMPLEXITY: Let's say we have n hotels in total. We start from j = 0, which would generally give us the biggest penalty, since we don't stop at any hotels except for hotel n. At this step, we calculate the penalty and remember it as minimum. We then move to j = 1 where we calculate the penalty from 0 to j = 1, since there are no other stops in between, and penalty from j to n. If the penalty is smaller than before, we add hotel j to the list and remember the new minimum. We then choose j = 2, we need to know penalty from 0 to 1 (which we already know), from 1 to 2 and from 0 to 2. We need to calculate the last 2 and choose the minimum out of p(0->1) + p(1->2) and p(0->2). We then also calculate p(2->n) and add it to the overall minimum.

For j >= 3 we continue this process, each time calculating exactly 3 penalties and comparing 2. The overall complexity of this algorithm is therefore $O(n)$.

Question 3

③ $G = (V, E)$

a)

$v_1 \rightarrow v_2 \rightarrow v_3 \rightarrow v_4 \rightarrow v_5$

$v_1 \rightarrow v_3 \rightarrow v_5$

$v_1 \rightarrow v_5$

(a)

The given algorithm would find the path $(v_1, v_2), (v_2, v_5)$ in our example, but the correct answer would be: $(v_1, v_3), (v_3, v_4), (v_4, v_5)$.

(b)

```
In [1]: """ graph1 = [
        ['A', ['B', 'C']],
        ['B', ['E']],
        ['C', ['D']],
        ['D', ['E']],
        ['E', []]
        ] """

graph2 = {
    'A': {'B', 'C'},
    'B': {'E'},
    'C': {'D'},
    'D': {'E'},
    'E': set()
}

""" IDEA: Start from vertex v1 and for every neighbour of v1, calculate the max path from neighbour to vn. """
def find_longest_path(graph, v1, vn, memo={}):    #we're using top down with memoization
    if v1 in memo:
        return memo[v1]
    if v1 == vn:    #base case
        return 0

    max_L = 0
    for neighbour in graph[v1]:
        path_L = find_longest_path(graph, neighbour, vn, memo)
        if path_L >= 0:
            max_L = max(max_L, 1 + path_L)
    memo[v1] = max_L    #because we're remembering the longest path from that node, we don't need to keep calculating it :)
    return max_L

print(find_longest_path(graph2, 'A', 'E'))

3
```

ANALYSIS: We cover the base case in case $v_1 = v_n$. Otherwise if there exists a neighbour of v_1 that is not v_n , we calculate the longest path from the neighbour to v_n recursively. On step j we take the maximum path to reach vertex j. Since we remember the previous longest distances (memoization), all the operations inside the for loop are constant, making complexity $O(n)$.

Question 4

```
In [2]: def subset_sum(a, t):
        n = len(a)
        dp = [[False] * (t + 1) for _ in range(n + 1)]    # dp[i][c] = whether we can reach sum c with first i elements

        for i in range(n + 1):    #base case bc we can make 0 if t is zero
            dp[i][0] = True

        for i in range(1, n + 1):    #O(n)
            for c in range(1, t + 1):    #O(t)
                if a[i - 1] >= c:    #if the element is already higher than c
                    dp[i][c] = dp[i - 1][c]    #then the optimal value is the optimal value of the previous elements
                else:    #either we don't take the ith element or we do:
                    dp[i][c] = dp[i - 1][c] or dp[i - 1][c - a[i - 1]]
        return dp[n][t]

        #we return true if we sum to the value t anytime during the iteration from first to last element
```

The algorithm is $O(tn)$ because we loop over all n elements and for each element we loop over t elements. In the end, we build $n * t$ big table.